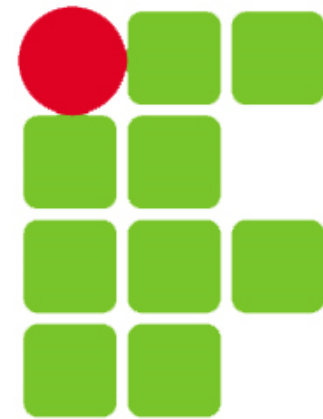


Desenvolvimento de Aplicações Móveis

Prof: Luis Nakamura

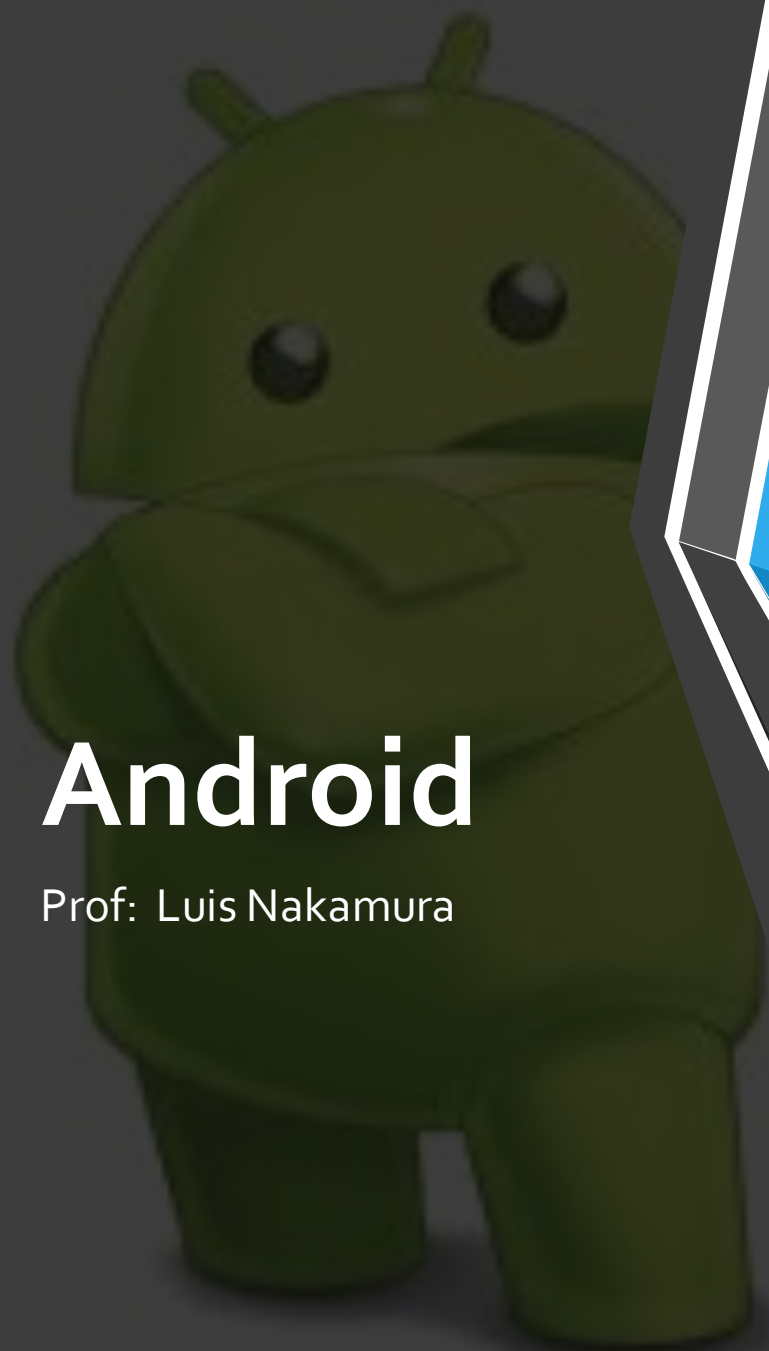


Parte 4

Apresentação da Aula 4

- **Objetivos da Aula:**

- Teoria: SQLite.
- Prática: Criando e utilizando o SQLite para armazenamento de dados local.



Android

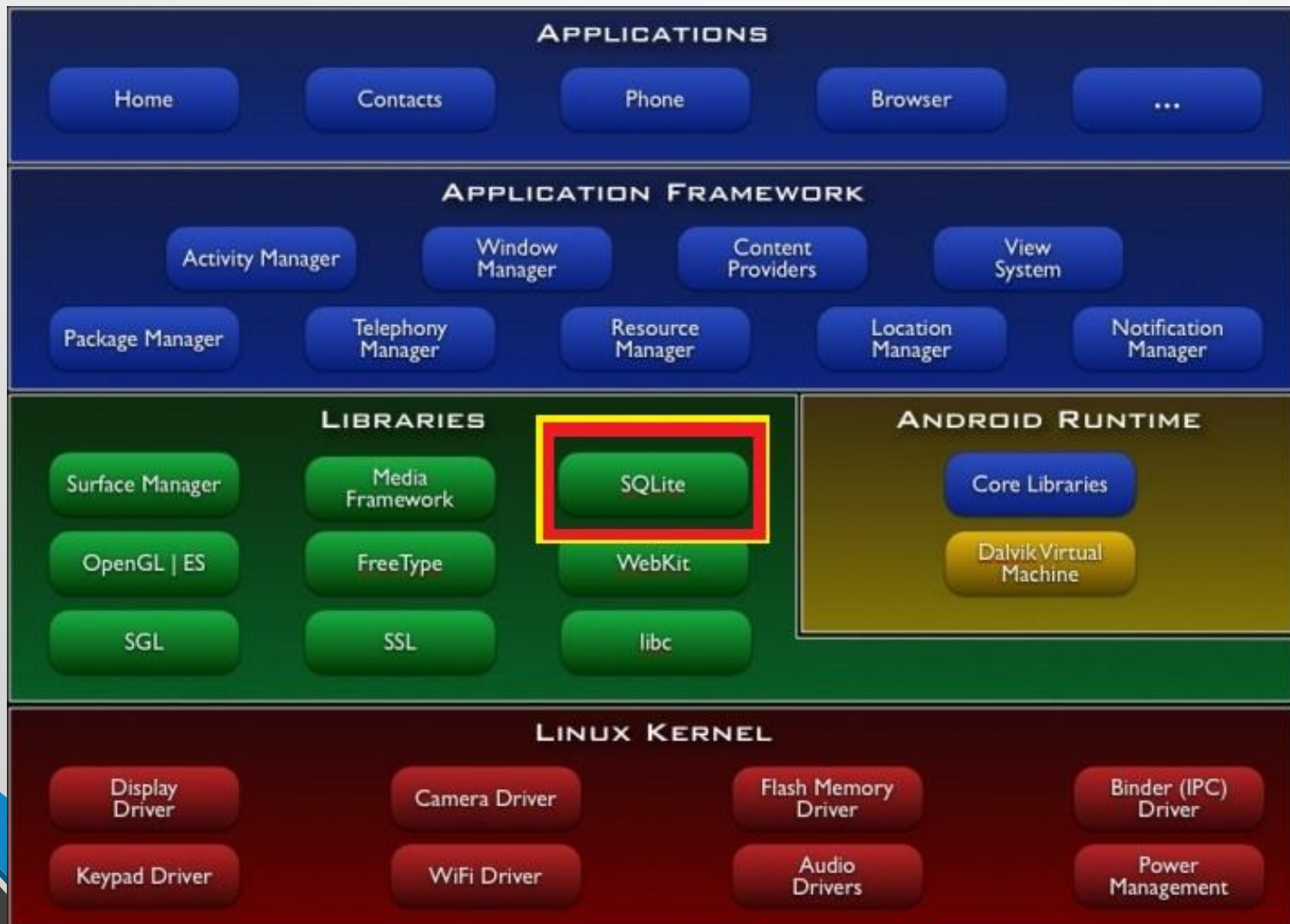
Prof: Luis Nakamura



Teoria: SQLite



- SQLite (Banco de Dados no Android)



Teoria: SQLite



- SQLite (Banco de Dados no Android)
 - O SQLite (www.sqlite.org) é uma biblioteca de software que implementa um banco de dados SQL transacional, auto-suficiente, sem servidor e de configuração zero.
 - Segundo seus desenvolvedores o SQLite é o banco de dados SQL mais amplamente difundido no mundo.
 - O código fonte para SQLite é do domínio público.

Teoria: SQLite



- SQLite (Banco de Dados no Android)
 - Para o uso SQLite utilizaremos o pacote:
 - `android.database.sqlite`
 - O banco será salvo em:
 - `/data/data/"pacote_app"/database`
 - Os tipos de dados aceitos no SQLite para Android são:

Tipo	Descrição
INTEGER	Inteiro (8 até 64 bits)
REAL	Ponto Flutuante (64bits)
TEXT	Textos (UTF-8, UTF-16)
BLOB	Array de Bytes

Teoria: SQLite



- SQLite (Banco de Dados no Android)
 - Há várias formas de se trabalhar com o SQLite, algumas classes merecem destaque:
 - **SQLiteOpenHelper**: é uma classe de “ajuda” usada para gerenciar a criação, a abertura e a atualização de bancos de dados.
 - **SQLiteDatabase**: tem métodos para criar, excluir e executar comandos SQL, e também executar outras tarefas comuns de gerenciamento de banco de dados.

Teoria: SQLite



- SQLite (Banco de Dados no Android)
- **SQLiteOpenHelper:**
 - .close: fecha a conexão com o BD;
 - .getReadableDatabase: cria ou abre o BD em modo apenas leitura.
 - .getWritableDatabase: cria (onCreate) ou abre (onOpen) o BD tanto para leitura como para escrita;
 - .getDatabaseName: retorna o nome do BD;
 - .onCreate: devemos sobrescrever esse método para criar o nosso banco de dados.
 - .onUpgrade: devemos sobrescrever esse método quando houver modificações no esquema do banco de dados ou quando houver uma nova versão.

Teoria: SQLite



- SQLite (Banco de Dados no Android)
- **SQLiteDatabase:**
 - .insert: insere no banco de dados;
 - .update: atualiza no banco de dados;
 - .query: seleciona (ler/recupera) do banco de dados, retorna um objeto Cursor;
 - .delete: remove do banco de dados.
 - .close: fecha a conexão com o banco de dados;
 - .execSQL: Executa um comando SQL no banco de dados, desde que NÃO RETORNE dados;

Teoria: SQLite



- **SQLite (Banco de Dados no Android)**
 - É interessante estender a classe `SQLiteOpenHelper` para utilizar os métodos que criam e atualizam o esquema do BD;
 - Devemos criar um construtor para essa nova classe que estende de `SQLiteOpenHelper` que passará para a classe pai o Contexto, o Nome do BD, uma fábrica de Cursor e a versão do BD;
 - Devemos também sobrescrever os métodos necessários como, por exemplo, o `onCreate` e o `onUpgrade`;
 - No método `onCreate` devemos utilizar o método `execSQL` de `SQLiteDatabase` para criar o nosso BD;

Prática: SQLite



```
public class DBOpenHelper extends SQLiteOpenHelper {  
  
    private static final String DB_NAME = "pessoas";  
    private static final int DB_VERSION = 1;  
  
    public DBOpenHelper(Context context) {  
        super(context, DB_NAME, null, DB_VERSION);  
    }  
  
    @Override  
    public void onCreate(SQLiteDatabase db) {  
  
        String createSQLDB = "CREATE TABLE pessoas " +  
            "(_id integer primary key autoincrement," +  
            "nome TEXT NOT NULL, email TEXT, tel TEXT, " +  
            "cep TEXT);";  
  
        db.execSQL(createSQLDB);  
  
    }  
}
```

Teoria: SQLite



- **SQLite (Banco de Dados no Android)**
 - Também devemos criar uma outra classe para realizar o CRUD dos nossos dados no BD;
 - Essa classe irá utilizar a classe anterior (DBOpenHelper) para criar/abrir o nosso BD;
 - Devemos implementar os métodos de abertura (open) e fechamento (close) do nosso BD.
 - Devemos implementar os métodos CRUD (Create/Insert, Read/Select, Update e Delete).
 - Lembrando que para inserir e atualizar dados usamos o objeto ContentValues que trabalha no conceito chave-valor.

Prática: SQLite



```
public class DBSQLiteAccess {  
  
    private DBOpenHelper dbHelper;  
    private SQLiteDatabase db;  
  
    public DBSQLiteAccess (Context context) {  
        dbHelper = new DBOpenHelper(context);  
    }  
  
    public void open() throws SQLException {  
        db = dbHelper.getWritableDatabase();  
    }  
  
    public void close() {  
        if(db != null) {  
            db.close();  
        }  
    }  
} //Continua...
```

Teoria e Prática: SQLite



- SQLite (Banco de Dados no Android)
 - Para facilitar a manipulação dos dados é interessante criar uma classe Pessoa com os atributos: (long id, String nome, String email, String tel, String cep) [Crie os gets and sets].

```
public class Pessoa {  
  
    private long id;  
    private String nome;  
    private String email;  
    private String tel;  
    private String cep;  
  
    public Pessoa(long id, String nome, String email, String tel,  
String cep) {  
        this.id = id;  
        this.nome = nome;  
        this.email = email;  
        this.tel = tel;  
        this.cep = cep;  
    }  
}
```

Prática: SQLite



- *Delete e Insert*

```
public void deletePessoa(long _id) {  
    open();  
    db.delete("pessoas", "_id=" + _id, null);  
    close();  
}  
  
public void insertPessoa(Pessoa p) {  
    ContentValues cValues = new ContentValues();  
    cValues.put("nome", p.getNome());  
    cValues.put("email", p.getEmail());  
    cValues.put("tel", p.getTel());  
    cValues.put("cep", p.getCep());  
  
    open();  
    db.insert("pessoas", null, cValues);  
    close();  
}
```

Prática: SQLite



- *SelectOne*

```
public Pessoa selectOnePessoa(long _id) {
    Pessoa p = null;
    String[] columns = new String[]{"_id", "nome", "email",
    "tel", "cep"};
    open();
    Cursor cursor = db.query("pessoas", columns, "_id="
    +_id, null, null, null, null);
    if(cursor.getCount() > 0) {
        cursor.moveToFirst();
        p = new Pessoa(    cursor.getLong(0),
                           cursor.getString(1),
                           cursor.getString(2),
                           cursor.getString(3),
                           cursor.getString(4));
    }
    close();
    return p;
}
```


Prática: SQLite



- *Update*

```
public void updatePessoa(Pessoa p) {  
  
    ContentValues cValues = new ContentValues();  
    cValues.put("nome", p.getNome());  
    cValues.put("email", p.getEmail());  
    cValues.put("tel", p.getTel());  
    cValues.put("cep", p.getCep());  
  
    open();  
    db.update("pessoas", cValues, "_id=" + p.getId(), null);  
    close();  
}
```

Prática: SQLite



- *SelectAll*

```
public ArrayList<Pessoa> selectAll() {  
  
    ArrayList<Pessoa> arrPessoas = new ArrayList<Pessoa>();  
    Pessoa p = null;  
  
    String[] columns = new String[]{"_id", "nome", "email",  
    "tel", "cep"};  
    open();  
    Cursor cursor = db.query("pessoas", columns, null, null,  
    null, null, "nome ASC");  
  
    if(cursor.getCount() > 0) {  
        cursor.moveToFirst();  
  
        //Continua...
```

Prática: SQLite



- *SelectAll*

```
//Continuacao..  
do {  
  
    p = new Pessoa( cursor.getLong(0),  
        cursor.getString(1),  
        cursor.getString(2),  
        cursor.getString(3),  
        cursor.getString(4));  
  
    arrPessoas.add(p);  
  
} while (cursor.moveToNext());  
}  
close();  
return arrPessoas;  
}
```

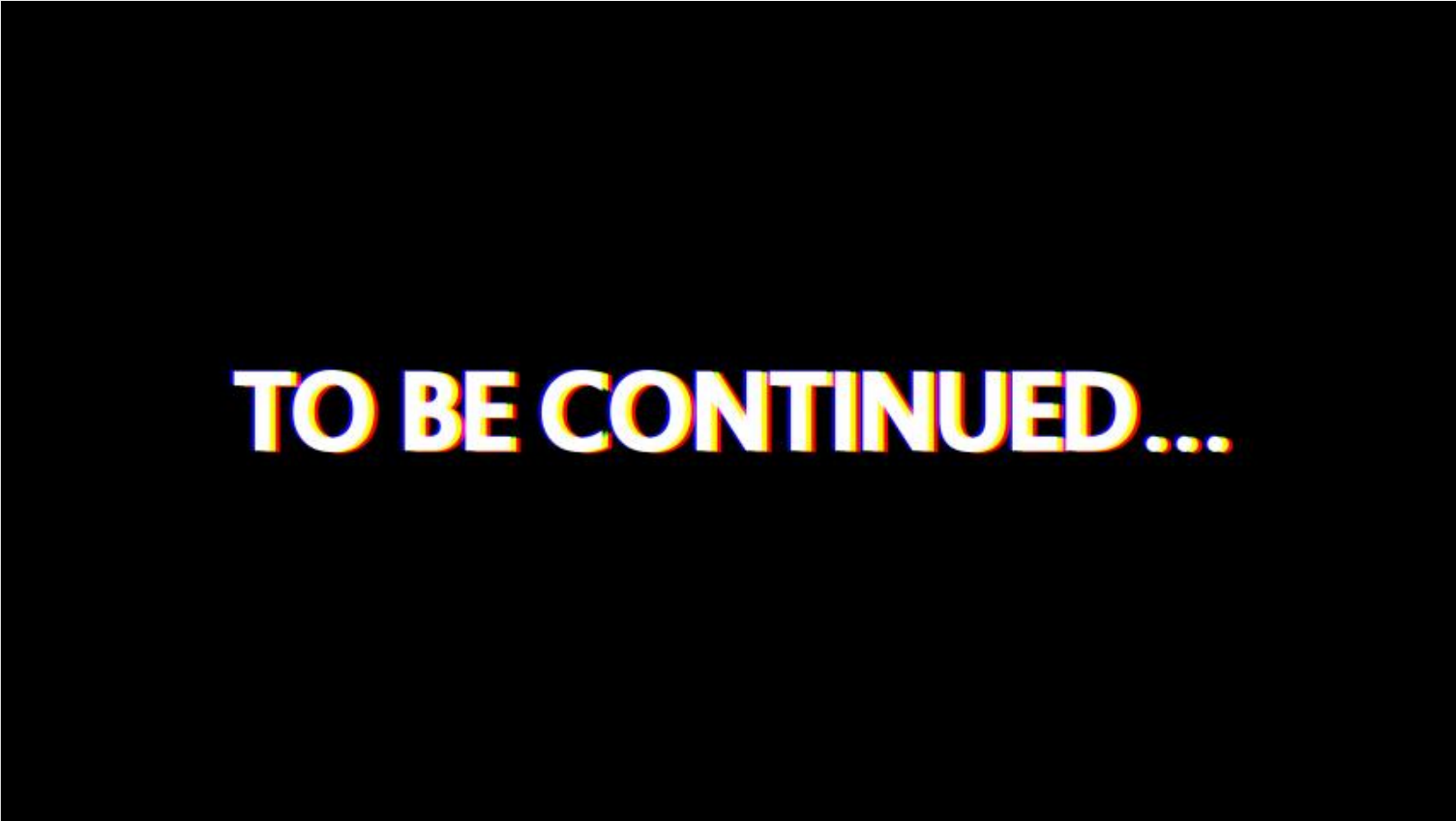
Prática: SQLite



- **SQLite (Banco de Dados no Android)**
 - Crie uma Activity com um ListView que exiba o ID e NOME de todas as Pessoas cadastradas no banco de dados (Use o método `SelectAll`);
 - Adicione um opção de Menu, para a criação de uma nova pessoa no BD. Essa opção deve abrir uma nova Activity de Cadastro (Use o método `insertPessoa`);
 - Crie um evento, para quando o usuário clicar sobre um item do ListView ele abra a tela de Cadastro em modo somente leitura.
 - Adicione dois items de Menu na Activity de Cadastro, um para Editar e outro para Excluir. Permita que o usuário atualize e exclua uma Pessoa.

Teoria: SQLite

- Continua....



TO BE CONTINUED...

Próximas Aulas

- Armazenamento de Dados (SQLite)
- Web Services
- Etc.

Referências

- <http://www.androidcentral.com/android-versions>
- <http://www.idc.com/prodserv/smartphone-os-market-share.jsp>
- http://www.cisco.com/c/en/us/solutions/collateral/service-provider/visual-networking-index-vni/white_paper_c11-520862.html
- <http://www.android.com/versions/kit-kat-4-4/>
- <http://www.android.com/versions/lollipop-5-0/>
- <http://developer.android.com/about/dashboards/index.html>
- DEITEL, P. J.; DEITEL, H. M.. Android para programadores: uma abordagem baseada em aplicativos. 1. ed. Porto Alegre: Bookman, 2012. 512p.
- FELKER, D.. Android application development for dummies. 1. ed. John Wiley & Sons, 2012. 456p.
- NUDELMAN, G.. Android design patterns: interaction designs. 1. ed. John Wiley & Sons, 2012. 384p.
- DARWIN, I. F.. Android cookbook. 1. ed. São Paulo: Novatec, 2012. 672p.